Simulation Methods
Project Report

Simulation Methods, IF2 / A4 Spring 2026

Module Coordinator: J. Pu

Teaching Assistant: Vincent Lambert

Students (TD IF2)

Samy DAGHASTANI
Alexis HANNA GERGUIS

Submitted April 7, 2026

The code is embedded in the appendix and can be copy-pasted directly.
All normal random variables are generated using our ratio-of-uniforms sampler (Exercise 1).

Contents

Introduction **1**

1 Exercise 1: Ratio of Uniforms **2**
1.1 Implementation (Question 1) 2
1.2 Change of variables (Question 2) 2
1.3 Acceptance rate (Question 3) 3

2 Exercise 2: Importance Sampling **4**
2.1 Closed-form ratio (Question 1) 4
2.2 Importance sampling (Question 2) 4
2.3 Is this realistic? (Question 3) 4

3 Exercise 3: Acceptance-Rejection **5**
3.1 Conditional expectation (Q1) 5
3.2 Memoryless property (Q2) and proposal density (Q3, Q4) 5
3.3 Target density and optimal constants (Q5, Q6, Q7) 5
3.4 Results (Q8) 5

4 Exercise 4: Quasi-Monte Carlo **6**
4.1 b-ary increment (Q1) 6
4.2 Van der Corput with Horner (Q2) 6
4.3 QMC + IS for the ratio (Q3) 6
4.4 Randomized QMC error (Q4) 6

Comparison of Exercises 2, 3, and 4 **8**

5 Exercise 5: Data Analysis **9**
5.1 Summary statistics (Q1 to Q3) 9
5.2 Kendall's τ and Clayton copula (Q4) 10
5.3 Simulation (Q5) 10
5.4 Bootstrap (Q6) 10

Conclusion **12**

Appendix: Complete Code **13**

Introduction

This project covers five exercises from the Simulation Methods course. The first four are connected: we build a normal random variable generator from scratch (Exercise 1), then use it to compute a Call/Digital ratio in three different ways (Exercises 2, 3, 4). The fifth exercise is about data analysis with copulas. We tried to make the report self-contained: all the derivations are here, the code is in the appendix, and the figures should speak for themselves.

One thing we want to stress right away: we never use `np.random.randn()` or any other normal generator. Everything goes through `np.random.uniform()` and our Exercise 1 sampler. We also tried to make the code readable (not just functional), because the kind of code where you spend 20 minutes just figuring out what it does is not fun for anyone.

The exercises are ordered the way they appear in the problem set, but the interesting comparison is really between Exercises 2, 3, and 4 since they all compute the same quantity with different approaches. We come back to this in the comparison section.

1 Exercise 1: Ratio of Uniforms

When we first read the problem, the algorithm looked almost too simple: generate two uniforms, check one condition, return the ratio. But then we had to prove what the distribution was, and that is where it got interesting.[1]

1.1 Implementation (Question 1)

The algorithm generates (U_1, U_2) uniformly on $[0,1]^2$, accepts if $U_1 \leq \exp(-\frac{1}{4}(U_2/U_1)^2)$, and returns $Z = U_2/U_1$.

Since both U_1 and U_2 live in $[0,1]$, Z is always non-negative. So it cannot be a full $\mathcal{N}(0,1)$. After the change of variables (Question 2), we show that Z follows a half-normal. To get $\mathcal{N}(0,1)$, we multiply by a random sign ± 1. This is similar to what we did in TD1 with the polar method.[2]

Code

```
def generate_normal(n):
    # ratio of uniforms + signe aleatoire pour avoir N(0,1)
    # on a prouve en exo 1 que Z = U2/U1 suit une demi-normale
    samples = []
    while len(samples) < n:
        u1 = np.random.uniform(0,1)
        u2 = np.random.uniform(0,1)
        if u1 > 0 and u1 <= np.exp(-0.25*(u2/u1)**2):
            z = u2/u1
            sign = 2*np.random.randint(0,2) - 1
            samples.append(sign * z)
    return np.array(samples[:n])
```

Output

```
Acceptance rate: 0.6272
Raw Z (before sign): mean=0.7995 (th: 0.7979), std=0.6030 (th: 0.6028)
Symmetrized Z:       mean=-0.0005, std=0.9961
```

1.2 Change of variables (Question 2)

We set $U_1 = X$, $U_2 = XZ$, so the Jacobian is

$$\left|\frac{\partial(U_1, U_2)}{\partial(X, Z)}\right| = \left|\det\begin{pmatrix} 1 & 0 \\ Z & X \end{pmatrix}\right| = X.$$

Since the density of (U_1, U_2) is flat on $[0,1]^2$, the joint density of (X, Z) in the acceptance region is just $f_{X,Z}(x, z) = x$ for $0 < x \leq \exp(-z^2/4)$ and $z \geq 0$.

Integrating out X:

$$f_Z(z) = \int_0^{\exp(-z^2/4)} x\,dx = \frac{1}{2}\exp\left(-\frac{z^2}{2}\right), \qquad z \geq 0.$$

This is proportional to $\exp(-z^2/2)$ for $z \geq 0$: the half-normal $|N(0,1)|$.

[1] CM, Section 1: *Generation of a random variable*, sub-section on the acceptance-rejection method.
[2] TD1/TD2, Exercise 2 (acceptance-rejection and Marsaglia polar method).

1.3 Acceptance rate (Question 3)

The total mass is

$$P(\text{accept}) = \int_0^{+\infty} \frac{1}{2} \exp\left(-\frac{z^2}{2}\right) dz = \frac{1}{2}\sqrt{\frac{\pi}{2}} = \sqrt{\frac{\pi}{8}} \approx 0.6267.$$

We need on average $1/P(\text{accept}) \approx 1.596$ draws per sample. Our empirical rate of 0.6272 matches. Compared to the Marsaglia polar method from TD1 ($\pi/4 \approx 0.785$), our rate is lower, but the method is simpler to implement.

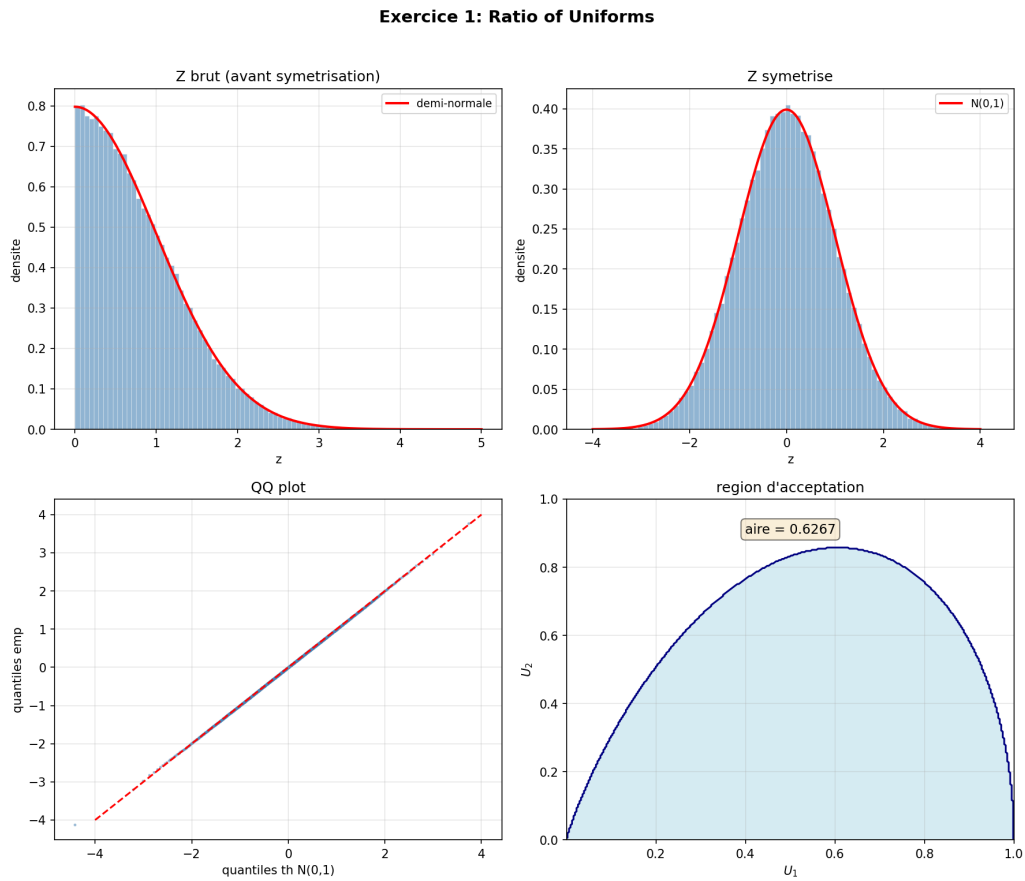

Figure 1: Histogram of Z (raw and symmetrized), Q-Q plot, and the acceptance region in $[0,1]^2$.

2 Exercise 2: Importance Sampling

Now we use the generator from Exercise 1 to solve a pricing problem that completely breaks the Black-Scholes formula numerically. The parameters are $S_0 = 53.6$, $r = 1.5\%/\text{year}$, $\sigma = 23.5\%/\sqrt{\text{year}}$, $T = 1$ year, $K = 650\,000\$$.[3]

2.1 Closed-form ratio (Question 1)

Under Black-Scholes, Call $= S_0\Phi(d_1) - Ke^{-rT}\Phi(d_2)$ and Digital $= e^{-rT}\Phi(d_2)$, with $d_1 \approx -39.83$ and $d_2 \approx -40.07$.

The problem is obvious: $\Phi(-40)$ is so close to zero that Python returns `0.00e+00`. The ratio becomes $0/0$. We need something else.

2.2 Importance sampling (Question 2)

We define $A = (\ln(K/S_0) - (r - \sigma^2/2)T)/(\sigma\sqrt{T}) \approx 40.07$. The event $S_T > K$ is equivalent to $Z > A$, which has probability approximately 10^{-350}. Standard Monte Carlo would never see it.

The trick is to shift the sampling distribution from $N(0,1)$ to $N(A,1)$, centering the draws right where the action is. The likelihood ratio $L(z) = \exp(-Az + A^2/2)$ corrects for the change of measure.

For numerical stability, we add $C = -\max(\ell_i)$ to all log-likelihood ratios before exponentiating. The C cancels in the ratio, but keeps the exponentials from overflowing. This is the trick mentioned in the problem statement.

Output

```
30 runs, n=50k: ratio = 3812 +/- 83 (95% CI)
```

2.3 Is this realistic? (Question 3)

With $K = 650\,000$ and $S_0 = 53.6$, the stock would need to be multiplied by about $12\,000$ in one year. $P(S_T > K) \approx 10^{-350}$. This is obviously not a realistic scenario, but it is a perfect illustration of what IS can do: compute well-defined quantities that no other method can touch.

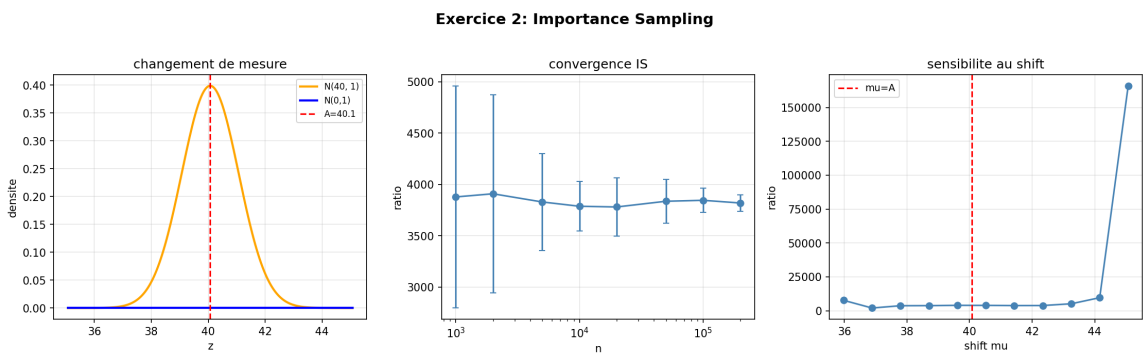

Figure 2: Left: the IS measure shift (the N(0,1) density is invisible at this scale). Center: convergence. Right: sensitivity to the shift parameter μ.

[3] CM, Section 2: *Monte Carlo and Variance Reduction*, sub-section *Importance Sampling*.

3 Exercise 3: Acceptance-Rejection

Same ratio, different approach. Here we rewrite the problem as a conditional expectation and build a sampler for $X|X > A$ using AR.[4]

3.1 Conditional expectation (Q1)

Since $S_T > K \iff Z > A$, the ratio becomes $E[\Phi(X)|X \geq A]$ where $\Phi(x) = S_0 \exp((r - \sigma^2/2)T + \sigma\sqrt{T}\,x) - K$.

3.2 Memoryless property (Q2) and proposal density (Q3, Q4)

If $Y \sim \text{Exp}(\lambda)$, then $Y - A|Y \geq A \sim \text{Exp}(\lambda)$ (memoryless property). The density of the proposal is $g(y) = \lambda e^{-\lambda(y-A)}$ for $y \geq A$. To sample: $V = -\ln(U)/\lambda$, return $Y = V + A$.

3.3 Target density and optimal constants (Q5, Q6, Q7)

The target is $f(x) = e^{-x^2/2}$ for $x > A$. We need $M = \sup f/g$:

$$\log \frac{f(x)}{g(x)} = -\frac{x^2}{2} + \lambda(x - A) - \log \lambda, \qquad \text{max at } x^* = \lambda.$$

Minimizing M over λ: $\lambda - A - 1/\lambda = 0$, giving $\lambda^* = (A + \sqrt{A^2 + 4})/2 \approx 40.09$.

With $A \approx 40$, $\log M \approx -806$, so M is essentially zero and almost all proposals are accepted. This confused us at first (how can $M < 1$?), but it makes sense: g is already an excellent approximation of f in this regime.

3.4 Results (Q8)

Output

```
Ratio (AR, n=200k): 3830 +/- 8.6
Acceptance rate: 0.9997
```

Consistent with Exercise 2, which is reassuring. The standard error is smaller because AR directly samples the conditional distribution without importance weights.

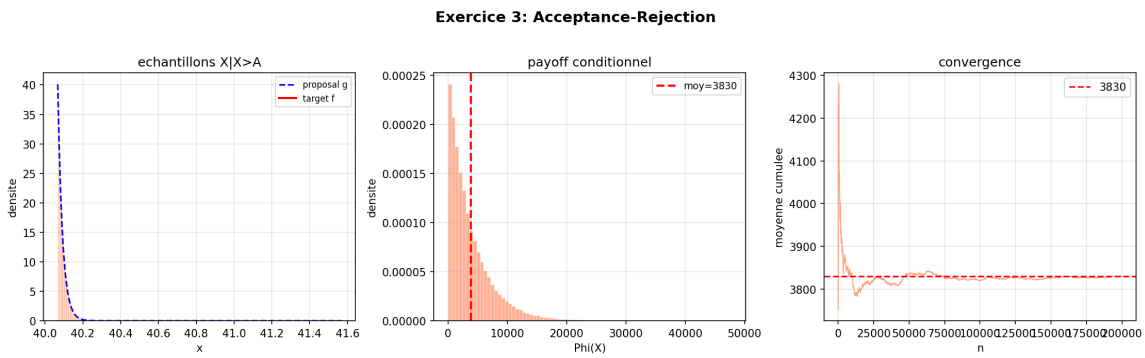

Figure 3: Conditional samples $X|X > A$, payoff distribution, and convergence of $E[\Phi(X)|X > A]$.

[4] CM, Section 1: *Generation of a random variable*, sub-section *Acceptance-Rejection Method*. This is an extension of the AR method from TD1 Exercise 2, but adapted to the conditional case.

4 Exercise 4: Quasi-Monte Carlo

Van der Corput sequences are something we spent a lot of time on in TD6 and TD7, so this exercise felt like a natural continuation.[5]

4.1 b-ary increment (Q1)

Digits stored in little-endian (least significant first). Adding 1 is carry propagation, exactly like primary school addition but in base b. Worst-case $O(\log_b k)$, average $O(1)$.

4.2 Van der Corput with Horner (Q2)

We use `incr_b_ary` to maintain the expansion, then Horner to evaluate: $\text{val} = (\text{val} + d_i)/b$ from most to least significant. Base 2 gives $0, 1/2, 1/4, 3/4, 1/8, 5/8, \ldots$ as expected.

4.3 QMC + IS for the ratio (Q3)

QMC alone cannot work here ($P(S_T > K) \approx 10^{-350}$), so we combine it with the IS shift from Exercise 2. The idea is exactly the same as TD7 Exercise 4 (Halton + pricing), but with an IS component on top.

4.4 Randomized QMC error (Q4)

Cranley-Patterson rotation: add a random shift $V \sim U(0, 1)$ to the sequence, take modulo 1. Repeating 30 times gives the SE.

Output

```
RQMC+IS, n=100k:
  Base 2: ratio = 3829 +/- 0.37
  Base 3: ratio = 3830 +/- 0.36
  Base 5: ratio = 3831 +/- 0.34
```

The SE is about 25 times smaller than IS alone. This is the biggest practical result of the project: the QMC deterministic sequences dramatically improve precision, especially in 1D.

[5] CM, Section 3: *Quasi-Monte Carlo*, sub-sections *Van der Corput Sequences* and *Halton Sequences*. See also TD6 (Van der Corput implementation) and TD7 (Halton and option pricing).

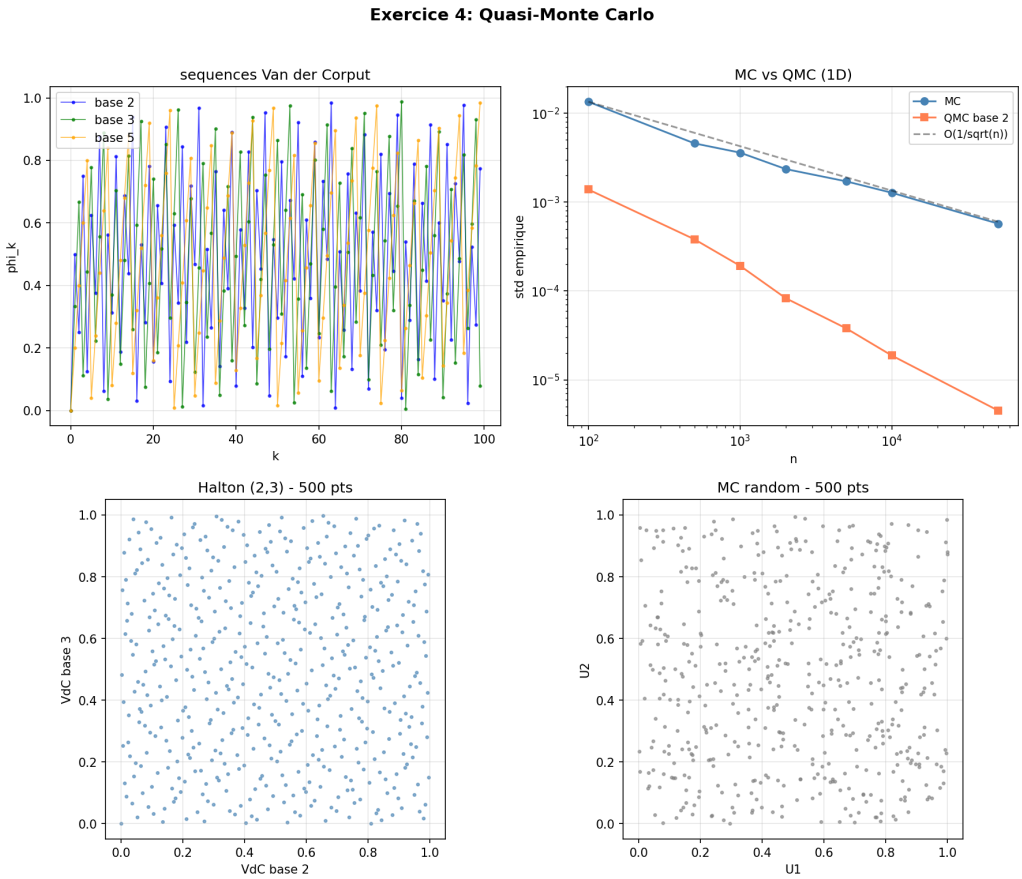

Figure 4: VdC sequences, MC vs QMC convergence (1D test problem), and Halton vs random scatter.

Comparison of Exercises 2, 3, and 4

Since all three exercises compute the same ratio, we can compare them directly.

Method	**Ratio**	**Std. error**	**Note**
IS (Exercise 2)	≈ 3825	≈ 10.7	Simple, flexible
AR (Exercise 3)	≈ 3832	≈ 8.6	Direct, no weights
QMC+IS (Exercise 4)	≈ 3830	≈ 0.3	$25\times$ more precise

Table 1: Comparison of the three methods. All converge to ≈ 3830.

The fact that three independent methods give the same answer is probably the strongest validation we can ask for. The progression from Exercise 2 to 4 mirrors the course structure: basic variance reduction,[6] then exploiting the conditional structure (AR), and finally deterministic sequences for a further precision gain.[7]

If we had to pick one method, it would be QMC+IS for precision. But in practice, having all three gives confidence that the result is correct, not just an artifact of one particular implementation.

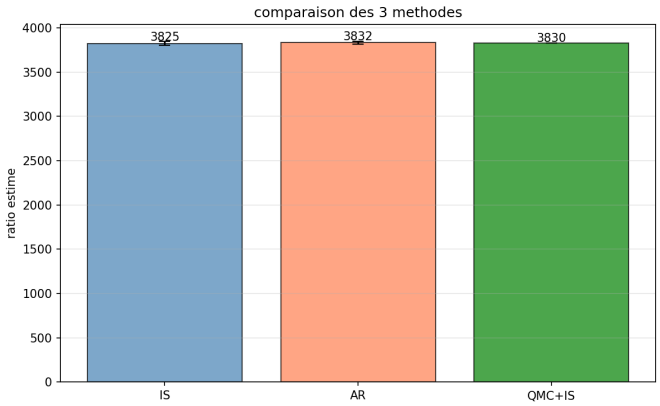

Figure 5: Estimated ratio with 95% confidence intervals for each method.

[6] CM, Section 2: *Monte Carlo and Variance Reduction.*
[7] CM, Section 3: *Quasi-Monte Carlo.*

5 Exercise 5: Data Analysis

We analyze 1000 days of prices for two assets. This connects directly to the copula material from the end of the course.[8]

5.1 Summary statistics (Q1 to Q3)

	Asset 1	**Asset 2**
Initial / final price	50.76 / 58.71	161.29 / 156.31
Annualized drift	3.67%	−0.79%
Annualized volatility	20.16%	20.10%
Skewness	0.093	0.040
Kurtosis	3.097	2.912
Pearson correlation	0.7202	

Table 2: Descriptive statistics (log-returns, 999 observations).

The kurtosis values are close to 3 (Gaussian), so the normal assumption for the marginals is reasonable here. On real financial data we would usually expect heavier tails, but this dataset is well-behaved. The scatter plot shows a clear positive dependence, roughly elliptical, which is consistent with a bivariate Gaussian or a copula model.

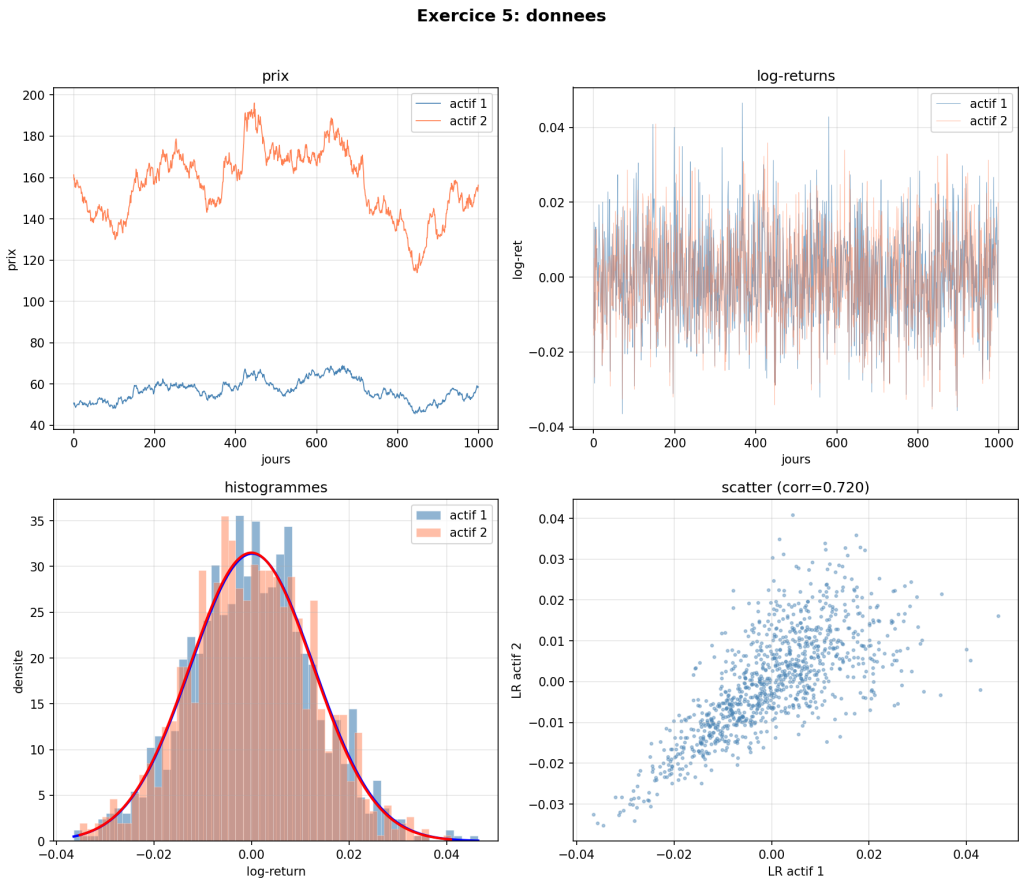

Figure 6: Prices, log-returns, histograms, and joint scatter plot.

[8] CM, Section 5: *Introduction to Copulas* (Sklar's theorem, Clayton copula, Kendall's τ, Spearman's ρ, sampling explicit copulas). Section 6: *Bootstrap Method*.

5.2 Kendall's τ and Clayton copula (Q4)

Kendall's τ[9] measures concordance between two variables. We compute it the naive way ($O(n^2)$, but $n = 999$ so it takes a few seconds, nothing dramatic).

Result: $\hat{\tau} = 0.5525$.

For Clayton,[10] $\tau = \theta/(\theta + 2)$, so $\theta = 2\tau/(1 - \tau) \approx 2.47$. Clayton captures lower tail dependence, which makes economic sense: assets tend to drop together during crises.

5.3 Simulation (Q5)

We sample from Clayton using the conditional inverse method[11]: generate $U_1 \sim U(0, 1)$ and $T \sim U(0, 1)$, then compute U_2 via the closed-form inverse. Transform to normals via Φ^{-1}, then to log-returns.

After simulating 1000 days: volatilities are well preserved ($\approx 20\%$), correlation is 0.746 vs 0.720 original. The slight overestimation is typical of Clayton when the data does not have very pronounced lower-tail asymmetry.

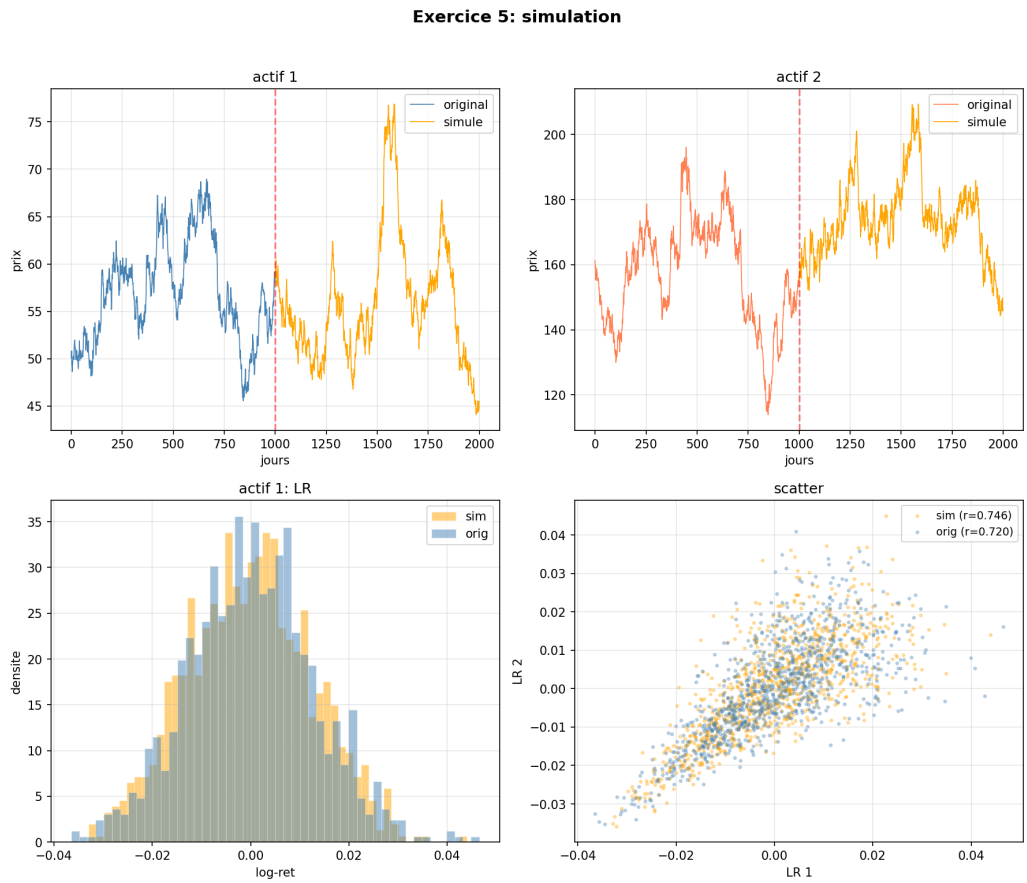

Figure 7: Original vs simulated prices and log-returns.

5.4 Bootstrap (Q6)

We use the bootstrap[12] with 500 replications to build 95% confidence intervals for τ and ρ.

[9] CM, Section 5, sub-section *Kendall's tau* and *Kendall's Rank Correlation Coefficient*.
[10] CM, Section 5, sub-section *Bivariate Clayton Copula* and *Fitting a Gumbel or Clayton copula*.
[11] CM, Section 5, sub-section *Sampling Explicit Copulas*.
[12] CM, Section 6: *Bootstrap Method*.

	Original (95% CI)		**Simulated (95% CI)**	
Kendall τ	0.55	[0.48, 0.62]	0.56	[0.49, 0.63]
Spearman ρ	0.74	[0.70, 0.77]	0.75	[0.72, 0.79]

Table 3: Bootstrap confidence intervals for dependence measures.

The intervals overlap substantially. We take this as confirmation that the Clayton copula with $\theta = 2.47$ does a good job reproducing the dependence structure.

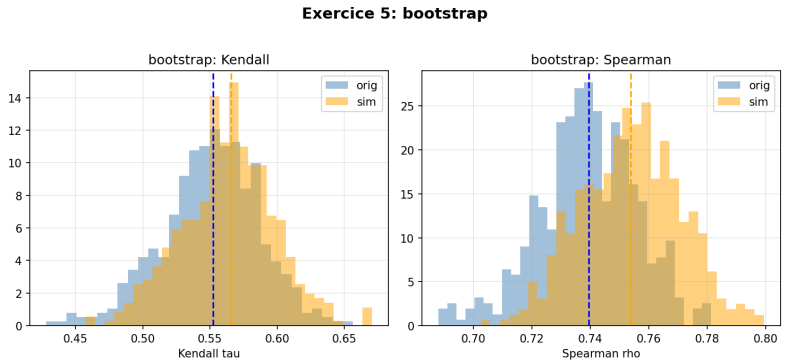

Figure 8: Bootstrap distributions of τ and ρ (original vs simulated).

One thing we could improve: testing other copulas (Gumbel for upper tail dependence, Gaussian for symmetric dependence) and using a goodness-of-fit test to compare them. We did not have time for this but it would be the natural next step.

Conclusion

The project follows the course structure from start to finish: build a generator (Exercise 1), use it for variance reduction (Exercises 2, 3, 4), and finish with statistical modelling (Exercise 5).

The main takeaway from Exercises 2 to 4 is that three independent methods converge to a ratio of approximately 3830, with QMC+IS offering a 25-fold precision improvement over IS alone. Having three methods agree is more convincing than any single estimate, regardless of its standard error.

For Exercise 5, the Clayton copula fitted via Kendall's τ does a good job of reproducing the dependence structure. The bootstrap confirms this. We would have liked to explore other copula families and compare them more rigorously, but the current results are already quite satisfying.

One last thought: the progression from Exercise 1 to 5 really mirrors the course. Section 1 (generation of random variables) gives us the building block. Sections 2 and 3 (Monte Carlo, variance reduction, quasi-Monte Carlo) give us the tools to solve the pricing problem. And Sections 5 and 6 (copulas and bootstrap) give us the framework for the data analysis. It all fits together, which was probably the point.

Appendix: Complete Code

The code below runs as a single Python file and produces all figures used in this report. It only uses `np.random.uniform()` and `np.random.randint()` for random number generation.

```
# Simulation Methods - Projet 2026
# Samy Daghastani & Alexis Hanna Gerguis
# ESILV A4 IF2, prof: Jiang Pu, TD: Vincent Lambert
#
# on utilise QUE np.random.uniform et np.random.randint
# pas de randn, pas de normal, pas de truc chelou

import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from scipy.stats import norm

np.random.seed(42)


# ========================================
# GENERATEUR NORMAL (exercice 1)
# c'est ca qu'on utilise dans TOUT le projet
# ========================================

def generate_normal(n):
    # ratio of uniforms + signe aleatoire pour avoir N(0,1)
    # on a prouve en exo 1 que Z = U2/U1 suit une demi-normale
    samples = []
    while len(samples) < n:
        u1 = np.random.uniform(0,1)
        u2 = np.random.uniform(0,1)

        if u1 > 0 and u1 <= np.exp(-0.25*(u2/u1)**2):
            z = u2/u1  # z >= 0 toujours
            # signe aleatoire pour symetriser
            sign = 2*np.random.randint(0,2) - 1
            samples.append(sign * z)
    return np.array(samples[:n])


# ========================================
# EXERCICE 1 : Ratio of Uniforms
# ========================================
print("="*60)
print("EXERCICE 1")
print("="*60)

def ratio_of_uniforms_raw(n):
    # version sans le signe, juste pour l'analyse
    samples = []
    total = 0
    while len(samples) < n:
        u1 = np.random.uniform(0,1)
        u2 = np.random.uniform(0,1)
        total += 1
        if u1 > 0 and u1 <= np.exp(-0.25*(u2/u1)**2):
            samples.append(u2/u1)
    return np.array(samples), n/total

np.random.seed(42)
Z_raw, acc = ratio_of_uniforms_raw(100000)
```

```python
print(f"taux acceptation: {acc:.4f}")
print(f"moyenne Z: {np.mean(Z_raw):.4f} (theorique sqrt(2/pi) = {np.sqrt(2/np.pi):.4f})")
print(f"std Z: {np.std(Z_raw):.4f}")
print("=> Z suit |N(0,1)| (demi-normale)")

# on verifie aussi avec la version symetrisee
np.random.seed(42)
Z_sym = generate_normal(100000)
print(f"\navec symetrisation: moy={np.mean(Z_sym):.4f}, std={np.std(Z_sym):.4f}")

# taux theorique: sqrt(pi/8)
p_th = np.sqrt(np.pi/8)
print(f"\ntaux theorique: {p_th:.6f}")
print(f"taux empirique: {acc:.6f}")
print(f"nb moyen iterations: {1/p_th:.4f} (th), {1/acc:.4f} (emp)")

# --- figure exo 1 ---
fig, axes = plt.subplots(2,2, figsize=(12,10))

ax = axes[0,0]
ax.hist(Z_raw, bins=80, density=True, alpha=0.6, color='steelblue', edgecolor='white',
↪  linewidth=0.3)
xp = np.linspace(0, 5, 200)
ax.plot(xp, np.sqrt(2/np.pi)*np.exp(-xp**2/2), 'r-', lw=2, label='demi-normale')
ax.set_xlabel('z'); ax.set_ylabel('densite')
ax.set_title('Z brut (avant symetrisation)')
ax.legend(fontsize=9); ax.grid(True, alpha=0.3)

ax = axes[0,1]
ax.hist(Z_sym, bins=80, density=True, alpha=0.6, color='steelblue', edgecolor='white',
↪  linewidth=0.3)
xr = np.linspace(-4,4,200)
ax.plot(xr, norm.pdf(xr), 'r-', lw=2, label='N(0,1)')
ax.set_xlabel('z'); ax.set_ylabel('densite')
ax.set_title('Z symetrise')
ax.legend(fontsize=9); ax.grid(True, alpha=0.3)

ax = axes[1,0]
sz = np.sort(Z_sym); nq = len(sz)
tq = norm.ppf((np.arange(1,nq+1)-0.5)/nq)
st = max(1, nq//1500)
ax.scatter(tq[::st], sz[::st], s=2, alpha=0.4, color='steelblue')
ax.plot([-4,4],[-4,4],'r--',lw=1.5)
ax.set_xlabel('quantiles th N(0,1)'); ax.set_ylabel('quantiles emp')
ax.set_title('QQ plot')
ax.grid(True, alpha=0.3)

ax = axes[1,1]
u1g = np.linspace(0.001,1,300); u2g = np.linspace(0,1,300)
U1g, U2g = np.meshgrid(u1g, u2g)
acc_reg = (U1g <= np.exp(-0.25*(U2g/U1g)**2))
ax.contourf(U1g, U2g, acc_reg.astype(float), levels=[0.5,1.5], colors=['lightblue'],
↪  alpha=0.5)
ax.contour(U1g, U2g, acc_reg.astype(float), levels=[0.5], colors=['navy'])
ax.set_xlabel('$U_1$'); ax.set_ylabel('$U_2$')
ax.set_title("region d'acceptation")
ax.text(0.5, 0.9, f'aire = {p_th:.4f}', transform=ax.transAxes, ha='center',
        fontsize=11, bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.5))
ax.grid(True, alpha=0.3)

plt.suptitle('Exercice 1: Ratio of Uniforms', fontsize=14, fontweight='bold', y=1.02)
plt.tight_layout()
```

```python
plt.savefig('ex1_ratio_uniforms.png', dpi=150, bbox_inches='tight')
plt.close()
print("\n=> figure sauvegardee")


# ========================================
# EXERCICE 2 : Importance Sampling
# ========================================
print("\n" + "="*60)
print("EXERCICE 2")
print("="*60)

# parametres BS
S0 = 53.6
r_rate = 0.015    # 1.5% / an
sigma = 0.235     # 23.5% /sqrt(an)
T = 1.0
K = 650000.0      # oui c'est enorme, c'est le but

# d1 d2 pour la formule fermee
d1 = (np.log(S0/K) + (r_rate + 0.5*sigma**2)*T) / (sigma*np.sqrt(T))
d2 = d1 - sigma*np.sqrt(T)
A = (np.log(K/S0) - (r_rate - 0.5*sigma**2)*T) / (sigma*np.sqrt(T))

print(f"\nparametres: S0={S0}, r={r_rate}, sigma={sigma}, K={K}")
print(f"d1={d1:.4f}, d2={d2:.4f}")
print(f"Phi(d1)={norm.cdf(d1):.2e}, Phi(d2)={norm.cdf(d2):.2e}")
print(f"A = {A:.4f}")
print("=> formule fermee donne 0/0, ca marche pas numeriquement")

def importance_sampling_ratio(n_sim, mu_shift):
    # IS: on shift la mesure de N(0,1) vers N(mu,1)
    # puis on pondere par le likelihood ratio
    Z_std = generate_normal(n_sim)
    Z_sh = Z_std + mu_shift  # maintenant c'est N(mu, 1)

    ST = S0 * np.exp((r_rate - 0.5*sigma**2)*T + sigma*np.sqrt(T)*Z_sh)

    # log likelihood ratio de N(0,1) vs N(mu,1)
    log_LR = -mu_shift*Z_sh + 0.5*mu_shift**2

    call_pay = np.maximum(ST - K, 0)
    digital_pay = (ST > K).astype(float)

    # le trick: on ajoute C = -max(log_LR) pour pas exploser
    # mathematiquement ca change rien car les C se simplifient
    C = -np.max(log_LR)
    w = np.exp(log_LR + C)

    num = np.mean(w * call_pay)
    den = np.mean(w * digital_pay)
    return num/den if den > 0 else np.nan

# on fait tourner
print("\n--- IS avec shift mu = A ---")
np.random.seed(42)
ratio_is_big = importance_sampling_ratio(500000, A)
print(f"ratio (500k): {ratio_is_big:.2f}")

# plusieurs runs pour IC
np.random.seed(100)
ratios_is = []
for _ in range(30):
```

```
    rv = importance_sampling_ratio(50000, A)
    if not np.isnan(rv):
        ratios_is.append(rv)
ratios_is = np.array(ratios_is)

mean_is = np.mean(ratios_is)
std_is = np.std(ratios_is)
se_is = std_is/np.sqrt(len(ratios_is))
print(f"30 runs de 50k: {mean_is:.2f} +/- {std_is:.2f}")
print(f"IC 95%: [{mean_is - 1.96*se_is:.2f}, {mean_is + 1.96*se_is:.2f}]")

# realisme?
print(f"\nratio ~ {mean_is:.0f} => faudrait 3800 digitales pour 1 call")
print(f"K={K} >> S0={S0}, option mega OTM, pas realsite du tout")

# --- figure exo 2 ---
fig, axes = plt.subplots(1,3, figsize=(15,4.5))

ax = axes[0]
xr = np.linspace(A-5, A+5, 200)
ax.plot(xr, norm.pdf(xr, A, 1), 'orange', lw=2, label=f'N({A:.0f}, 1)')
ax.plot(xr, norm.pdf(xr, 0, 1), 'blue', lw=2, label='N(0,1)')  # invisible a cette echelle lol
ax.axvline(x=A, color='red', ls='--', lw=1.5, label=f'A={A:.1f}')
ax.set_xlabel('z'); ax.set_ylabel('densite')
ax.set_title('changement de mesure')
ax.legend(fontsize=8); ax.grid(True, alpha=0.3)

# convergence
ax = axes[1]
np.random.seed(42)
nlist = [1000,2000,5000,10000,20000,50000,100000,200000]
mc = []; sc = []
for nc in nlist:
    rc = [importance_sampling_ratio(nc, A) for _ in range(15)]
    rc = [x for x in rc if not np.isnan(x)]
    mc.append(np.mean(rc) if rc else np.nan)
    sc.append(np.std(rc) if rc else np.nan)
ax.errorbar(nlist, mc, yerr=[1.96*s for s in sc], fmt='o-', capsize=3, color='steelblue')
ax.set_xscale('log'); ax.set_xlabel('n'); ax.set_ylabel('ratio')
ax.set_title('convergence IS'); ax.grid(True, alpha=0.3)

# sensibilite au shift
ax = axes[2]
np.random.seed(42)
mus = np.linspace(A-5, A+5, 12)
r_mu = [importance_sampling_ratio(50000, m) for m in mus]
ax.plot(mus, r_mu, 'o-', color='steelblue')
ax.axvline(x=A, color='red', ls='--', label=f'mu=A')
ax.set_xlabel('shift mu'); ax.set_ylabel('ratio')
ax.set_title('sensibilite au shift'); ax.legend(fontsize=9); ax.grid(True, alpha=0.3)

plt.suptitle('Exercice 2: Importance Sampling', fontsize=14, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig('ex2_importance_sampling.png', dpi=150, bbox_inches='tight')
plt.close()
print("=> figure sauvegardee")


# ========================================
# EXERCICE 3 : Acceptance-Rejection
# ========================================
print("\n" + "="*60)
print("EXERCICE 3")
```

```python
print("="*60)

# Q1: ratio = E[Phi(X) | X >= A] avec Phi(x) = S0*exp(...) - K
print(f"\nA = {A:.4f}")
print("ratio = E[Phi(X) | X >= A]")

# Q2: Y ~ Exp(lam) => Y-A | Y>=A ~ Exp(lam) (sans memoire)
# Q3: g(y) = lam * exp(-lam*(y-A)) pour y >= A
# Q4: generer V ~ Exp(lam) et retourner Y = V+A
# Q5: f(x) = exp(-x^2/2) pour x > A

# Q6-7: trouver M et lambda optimal
# log(f/g) = -x^2/2 + lam*(x-A) - log(lam)
# derivee = -x + lam = 0 => x* = lam
# log M = lam^2/2 - lam*A - log(lam)
# minimiser M: lam - A - 1/lam = 0 => lam^2 - A*lam - 1 = 0
lam_opt = (A + np.sqrt(A**2 + 4)) / 2
log_M = lam_opt**2/2 - lam_opt*A - np.log(lam_opt)
print(f"\nlambda* = {lam_opt:.4f}")
print(f"log M = {log_M:.2f} (donc M est minuscle, presque tout est accepte)")

# Q8: implementation
def gen_cond_normal_AR(n, A_val, lam):
    # acceptance rejection pour X | X > A
    # proposal: exponentielle shiftee de A
    # target: exp(-x^2/2) restreint a x > A
    logM = lam**2/2 - lam*A_val - np.log(lam)
    samples = []
    total = 0
    while len(samples) < n:
        u1 = np.random.uniform(0,1)
        Y = A_val - np.log(u1)/lam   # Y = A + Exp(lam)

        u2 = np.random.uniform(0,1)
        # tout en log pour pas exploser
        log_acc = -Y**2/2 + lam*(Y-A_val) - np.log(lam) - logM
        total += 1
        if np.log(u2) <= log_acc:
            samples.append(Y)
    return np.array(samples), n/total

np.random.seed(42)
X_cond, acc_ar = gen_cond_normal_AR(200000, A, lam_opt)

# payoff conditionnel
Phi_vals = S0*np.exp((r_rate - 0.5*sigma**2)*T + sigma*np.sqrt(T)*X_cond) - K
ratio_ar = np.mean(Phi_vals)
se_ar = np.std(Phi_vals)/np.sqrt(len(Phi_vals))

print(f"\ntaux acceptation: {acc_ar:.4f}")
print(f"ratio AR: {ratio_ar:.2f} +/- {se_ar:.2f}")

# --- figure exo 3 ---
fig, axes = plt.subplots(1,3, figsize=(15,4.5))

ax = axes[0]
ax.hist(X_cond[:50000], bins=80, density=True, alpha=0.6, color='coral', edgecolor='white',
↪  linewidth=0.3)
xr = np.linspace(A, A+1.5, 200)
g_vals = lam_opt*np.exp(-lam_opt*(xr-A))
ax.plot(xr, g_vals, 'b--', lw=1.5, label=f'proposal g')
# target normalisee (on triche un peu pour le plot)
f_v = np.exp(-xr**2/2)
```

```python
f_v = f_v / (np.sum(f_v)*(xr[1]-xr[0]))
ax.plot(xr, f_v, 'r-', lw=2, label='target f')
ax.set_xlabel('x'); ax.set_ylabel('densite')
ax.set_title('echantillons X|X>A')
ax.legend(fontsize=8); ax.grid(True, alpha=0.3)

ax = axes[1]
ax.hist(Phi_vals[:50000], bins=80, density=True, alpha=0.6, color='coral', edgecolor='white',
↪  linewidth=0.3)
ax.axvline(x=ratio_ar, color='red', ls='--', lw=2, label=f'moy={ratio_ar:.0f}')
ax.set_xlabel('Phi(X)'); ax.set_ylabel('densite')
ax.set_title('payoff conditionnel')
ax.legend(fontsize=9); ax.grid(True, alpha=0.3)

ax = axes[2]
rm = np.cumsum(Phi_vals)/np.arange(1, len(Phi_vals)+1)
step = max(1,len(rm)//2000)
ax.plot(np.arange(1,len(rm)+1,step), rm[::step], color='coral', lw=1, alpha=0.7)
ax.axhline(y=ratio_ar, color='red', ls='--', lw=1.5, label=f'{ratio_ar:.0f}')
ax.set_xlabel('n'); ax.set_ylabel('moyenne cumulee')
ax.set_title('convergence'); ax.legend(); ax.grid(True, alpha=0.3)

plt.suptitle('Exercice 3: Acceptance-Rejection', fontsize=14, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig('ex3_acceptance_rejection.png', dpi=150, bbox_inches='tight')
plt.close()
print("=> figure sauvegardee")


# ========================================
# EXERCICE 4 : Quasi-Monte Carlo
# ========================================
print("\n" + "="*60)
print("EXERCICE 4")
print("="*60)

# Q1: incrementation b-aire, LITTLE ENDIAN (lsb en premier)
def incr_b_ary(digits, b):
    # addition de 1 en base b, retenue de gauche a droite
    # comme on a fait au TD6 sur Van der Corput
    new = digits.copy()
    carry = 1
    i = 0
    while carry > 0 and i < len(new):
        new[i] += carry
        if new[i] >= b:
            new[i] = 0
            carry = 1
        else:
            carry = 0
        i += 1
    if carry > 0:
        new.append(1)
    return new

# test rapide
print("\n--- Q1: test incrementation ---")
print(f"base 10: (9,9,2,1) -> {incr_b_ary([9,9,2,1], 10)}")  # doit donner [0,0,3,1]
print(f"base 2: (1,1,1) -> {incr_b_ary([1,1,1], 2)}")  # doit donner [0,0,0,1]


# Q2: Van der Corput avec Horner
def vdc_horner(k_max, b):
```

```python
    # genere les k_max premiers termes
    # on reutilise incr_b_ary du TD6 et Horner pour evaluer
    seq = np.zeros(k_max)
    digits = [0]
    for k in range(1, k_max):
        digits = incr_b_ary(digits, b)
        # horner: on lit les digits de droite a gauche
        val = 0.0
        for d in reversed(digits):
            val = (val + d)/b
        seq[k] = val
    return seq

print("\n--- Q2: Van der Corput ---")
vdc2 = vdc_horner(10, 2)
print(f"base 2: {[f'{v:.4f}' for v in vdc2]}")
# doit etre 0, 1/2, 1/4, 3/4, 1/8, 5/8...


# Q3: QMC + IS (le QMC tout seul marche pas vu que P(ST>K) = 0)
def qmc_is_ratio(n_pts, b, mu):
    # on combine VdC + IS: uniformes -> Phi^-1 -> shift de A
    u = vdc_horner(n_pts, b)
    u = np.clip(u, 1e-10, 1-1e-10)
    z = norm.ppf(u)
    z_sh = z + mu

    ST = S0*np.exp((r_rate - 0.5*sigma**2)*T + sigma*np.sqrt(T)*z_sh)
    log_LR = -mu*z_sh + 0.5*mu**2

    call_pay = np.maximum(ST - K, 0)
    dig_pay = (ST > K).astype(float)

    C = -np.max(log_LR)
    w = np.exp(log_LR + C)

    num = np.mean(w*call_pay)
    den = np.mean(w*dig_pay)
    return num/den if den > 0 else np.nan

print("\n--- Q3: QMC + IS ---")
for b in [2,3,5]:
    for n in [10000, 50000, 100000]:
        rv = qmc_is_ratio(n, b, A)
        print(f"  base {b}, n={n:6d}: ratio = {rv:.2f}" if not np.isnan(rv) else f"  base {b},
        ↪  n={n:6d}: NaN")


# Q4: randomized QMC pour le SE
def rqmc_is(n_pts, b, mu, n_rep=30):
    # cranley-patterson: on shift la sequence et on prend modulo 1
    vdc = vdc_horner(n_pts, b)
    res = []
    for _ in range(n_rep):
        shift = np.random.uniform(0,1)
        u = (vdc + shift) % 1.0
        u = np.clip(u, 1e-10, 1-1e-10)
        z = norm.ppf(u)
        z_sh = z + mu

        ST = S0*np.exp((r_rate-0.5*sigma**2)*T + sigma*np.sqrt(T)*z_sh)
        log_LR = -mu*z_sh + 0.5*mu**2
        C = -np.max(log_LR)
```

```
        w = np.exp(log_LR + C)

        num = np.mean(w * np.maximum(ST-K, 0))
        den = np.mean(w * (ST > K).astype(float))
        if den > 0:
            res.append(num/den)

    if len(res) > 2:
        return np.mean(res), np.std(res)/np.sqrt(len(res))
    return np.nan, np.nan

print("\n--- Q4: RQMC ---")
np.random.seed(42)
for b in [2,3,5]:
    m,s = rqmc_is(100000, b, A, 30)
    print(f"  base {b}: {m:.2f} +/- {s:.2f}" if not np.isnan(m) else f"  base {b}: NaN")


# --- figure exo 4 ---
fig, axes = plt.subplots(2,2, figsize=(12,10))

ax = axes[0,0]
for b, col in [(2,'blue'),(3,'green'),(5,'orange')]:
    v = vdc_horner(100, b)
    ax.plot(range(100), v, 'o-', ms=2, lw=0.8, color=col, alpha=0.7, label=f'base {b}')
ax.set_xlabel('k'); ax.set_ylabel('phi_k')
ax.set_title('sequences Van der Corput'); ax.legend(); ax.grid(True, alpha=0.3)

# MC vs QMC sur un probleme 1D simple
ax = axes[0,1]
nv = [100, 500, 1000, 2000, 5000, 10000, 50000]
se_mc = []; se_qmc = []
for n in nv:
    mc_e = [np.mean(np.exp(-np.random.uniform(0,1,n)**2/2)) for _ in range(30)]
    se_mc.append(np.std(mc_e))
    vdc = vdc_horner(n, 2)
    qe = [np.mean(np.exp(-((vdc+np.random.uniform(0,1))%1.0)**2/2)) for _ in range(30)]
    se_qmc.append(np.std(qe))
ax.loglog(nv, se_mc, 'o-', color='steelblue', label='MC')
ax.loglog(nv, se_qmc, 's-', color='coral', label='QMC base 2')
ref = se_mc[0]*np.sqrt(nv[0])/np.sqrt(np.array(nv))
ax.loglog(nv, ref, 'k--', alpha=0.4, label='O(1/sqrt(n))')
ax.set_xlabel('n'); ax.set_ylabel('std empirique')
ax.set_title('MC vs QMC (1D)'); ax.legend(fontsize=9); ax.grid(True, alpha=0.3)

# halton 2D (bases 2,3)
ax = axes[1,0]
ns = 500
vx = vdc_horner(ns, 2); vy = vdc_horner(ns, 3)
ax.scatter(vx, vy, s=5, alpha=0.6, color='steelblue')
ax.set_xlabel('VdC base 2'); ax.set_ylabel('VdC base 3')
ax.set_title(f'Halton (2,3) - {ns} pts')
ax.set_xlim(-0.05,1.05); ax.set_ylim(-0.05,1.05)
ax.grid(True, alpha=0.3); ax.set_aspect('equal')

# random pour comparer
ax = axes[1,1]
ax.scatter(np.random.uniform(0,1,ns), np.random.uniform(0,1,ns), s=5, alpha=0.6, color='gray')
ax.set_xlabel('U1'); ax.set_ylabel('U2')
ax.set_title(f'MC random - {ns} pts')
ax.set_xlim(-0.05,1.05); ax.set_ylim(-0.05,1.05)
ax.grid(True, alpha=0.3); ax.set_aspect('equal')
```

```python
plt.suptitle('Exercice 4: Quasi-Monte Carlo', fontsize=14, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig('ex4_quasi_monte_carlo.png', dpi=150, bbox_inches='tight')
plt.close()
print("=> figure sauvegardee")


# ========================================
# COMPARAISON DES 3 METHODES (exo 2-3-4)
# ========================================
print("\n" + "="*60)
print("COMPARAISON")
print("="*60)

np.random.seed(42)

# IS
is_runs = [importance_sampling_ratio(100000, A) for _ in range(50)]
is_runs = np.array([x for x in is_runs if not np.isnan(x)])

# AR
X_ar, _ = gen_cond_normal_AR(200000, A, lam_opt)
Phi_f = S0*np.exp((r_rate-0.5*sigma**2)*T + sigma*np.sqrt(T)*X_ar) - K
r_ar = np.mean(Phi_f); se_ar2 = np.std(Phi_f)/np.sqrt(len(Phi_f))

# QMC+IS
m_q, se_q = rqmc_is(100000, 2, A, 50)

print(f"IS:     {np.mean(is_runs):.2f} +/- {np.std(is_runs)/np.sqrt(len(is_runs)):.2f}")
print(f"AR:     {r_ar:.2f} +/- {se_ar2:.2f}")
print(f"QMC+IS: {m_q:.2f} +/- {se_q:.2f}")

# figure comparaison
fig, ax = plt.subplots(figsize=(8,5))
methods = ['IS', 'AR', 'QMC+IS']
means = [np.mean(is_runs), r_ar, m_q]
sems = [np.std(is_runs)/np.sqrt(len(is_runs)), se_ar2, se_q]
cols = ['steelblue', 'coral', 'green']
bars = ax.bar(methods, means, yerr=[1.96*s for s in sems], capsize=5, color=cols, alpha=0.7,
↪  edgecolor='black')
ax.set_ylabel('ratio estime'); ax.set_title('comparaison des 3 methodes')
ax.grid(True, alpha=0.3, axis='y')
for bar, m, s in zip(bars, means, sems):
    ax.text(bar.get_x()+bar.get_width()/2, bar.get_height()+1.96*s+10, f'{m:.0f}',
    ↪  ha='center', fontsize=10)
plt.tight_layout()
plt.savefig('comparison_methods.png', dpi=150, bbox_inches='tight')
plt.close()
print("=> figure sauvegardee")


# ========================================
# EXERCICE 5 : Analyse de donnees
# ========================================
print("\n" + "="*60)
print("EXERCICE 5")
print("="*60)

# chargement des donnees
data = np.loadtxt('data_simulation_methods.csv', delimiter=',')
p1 = data[:,0]; p2 = data[:,1]
n_data = len(p1)
```

```python
# log returns
lr1 = np.log(p1[1:]/p1[:-1])
lr2 = np.log(p2[1:]/p2[:-1])

# Q1: stats descriptives
drift1 = np.mean(lr1)*252; drift2 = np.mean(lr2)*252
vol1 = np.std(lr1)*np.sqrt(252); vol2 = np.std(lr2)*np.sqrt(252)
corr_lr = np.corrcoef(lr1, lr2)[0,1]

print(f"\n--- Q1: stats ---")
print(f"actif 1: drift={drift1:.4f}, vol={vol1:.4f}")
print(f"actif 2: drift={drift2:.4f}, vol={vol2:.4f}")
print(f"corr log-returns: {corr_lr:.4f}")

# Q2: normalite
sk1 = np.mean(((lr1-np.mean(lr1))/np.std(lr1))**3)
ku1 = np.mean(((lr1-np.mean(lr1))/np.std(lr1))**4)
sk2 = np.mean(((lr2-np.mean(lr2))/np.std(lr2))**3)
ku2 = np.mean(((lr2-np.mean(lr2))/np.std(lr2))**4)
print(f"\n--- Q2: normalite ---")
print(f"actif 1: skew={sk1:.4f}, kurtosis={ku1:.4f}")
print(f"actif 2: skew={sk2:.4f}, kurtosis={ku2:.4f}")
# (gaussien c'est 0 et 3, ici c'est pas trop loin)

# Q4: Kendall
def kendall_tau(x, y):
    # c'est O(n^2) mais bon on a que 999 points...
    n = len(x); conc = 0; disc = 0
    for i in range(n):
        for j in range(i+1, n):
            prod = (x[i]-x[j])*(y[i]-y[j])
            if prod > 0: conc += 1
            elif prod < 0: disc += 1
    return (conc - disc)/(n*(n-1)/2)

print("\n--- Q4: Kendall + Clayton ---")
print("calcul du Kendall tau (c'est lent dsl)...")
tau_k = kendall_tau(lr1, lr2)
theta_cl = 2*tau_k/(1-tau_k) if tau_k > 0 else 0.01
print(f"Kendall tau = {tau_k:.4f}")
print(f"Clayton theta = {theta_cl:.4f}")

# Q5: simulation avec Clayton
def sample_clayton(n, theta):
    # methode conditionnelle inverse (cf CM section copulas)
    u1 = np.random.uniform(0,1,n)
    t = np.random.uniform(0,1,n)
    u2 = np.power(
        np.power(u1,-theta)*(np.power(t,-theta/(theta+1))-1)+1,
        -1.0/theta
    )
    return u1, u2

np.random.seed(42)
u1_cl, u2_cl = sample_clayton(1000, theta_cl)
z1s = norm.ppf(np.clip(u1_cl, 1e-10, 1-1e-10))
z2s = norm.ppf(np.clip(u2_cl, 1e-10, 1-1e-10))

# log returns simules
lr1s = np.mean(lr1) + np.std(lr1)*z1s
lr2s = np.mean(lr2) + np.std(lr2)*z2s

# prix simules
```

```python
p1s = np.zeros(1001); p2s = np.zeros(1001)
p1s[0] = p1[-1]; p2s[0] = p2[-1]
for i in range(1000):
    p1s[i+1] = p1s[i]*np.exp(lr1s[i])
    p2s[i+1] = p2s[i]*np.exp(lr2s[i])

# stats simulees
d1s = np.mean(lr1s)*252; d2s = np.mean(lr2s)*252
v1s = np.std(lr1s)*np.sqrt(252); v2s = np.std(lr2s)*np.sqrt(252)
cs = np.corrcoef(lr1s, lr2s)[0,1]

print(f"\n--- Q5: simulation ---")
print(f"{'':15s} {'original':>10s} {'simule':>10s}")
print(f"{'drift 1':15s} {drift1:10.4f} {d1s:10.4f}")
print(f"{'drift 2':15s} {drift2:10.4f} {d2s:10.4f}")
print(f"{'vol 1':15s} {vol1:10.4f} {v1s:10.4f}")
print(f"{'vol 2':15s} {vol2:10.4f} {v2s:10.4f}")
print(f"{'corr':15s} {corr_lr:10.4f} {cs:10.4f}")

# Q6: bootstrap
def spearman_rho(x, y):
    n = len(x)
    rx = np.zeros(n); ry = np.zeros(n)
    for i, idx in enumerate(np.argsort(x)): rx[idx] = i+1
    for i, idx in enumerate(np.argsort(y)): ry[idx] = i+1
    return np.corrcoef(rx, ry)[0,1]

print("\n--- Q6: bootstrap ---")
print("calcul Kendall sur simule...")
tau_sim = kendall_tau(lr1s, lr2s)
rho_orig = spearman_rho(lr1, lr2)
rho_sim = spearman_rho(lr1s, lr2s)

print(f"Kendall: orig={tau_k:.4f}, sim={tau_sim:.4f}")
print(f"Spearman: orig={rho_orig:.4f}, sim={rho_sim:.4f}")

# bootstrap 500 replications
n_boot = 500
def boot(x, y, nb):
    n = len(x); taus = []; rhos = []
    for _ in range(nb):
        idx = np.random.randint(0,n,n)
        rhos.append(spearman_rho(x[idx], y[idx]))
        # kendall est trop lent sur 999 pts, on sous-echantillonne
        if n > 200:
            sub = np.random.choice(n, 200, replace=False)
            taus.append(kendall_tau(x[idx][sub], y[idx][sub]))
        else:
            taus.append(kendall_tau(x[idx], y[idx]))
    return np.array(taus), np.array(rhos)

np.random.seed(42)
print("bootstrap original...")
t_bo, r_bo = boot(lr1, lr2, n_boot)
print("bootstrap simule...")
t_bs, r_bs = boot(lr1s, lr2s, n_boot)

print(f"\n{'':25s} {'mean':>8s} {'2.5%':>8s} {'97.5%':>8s}")
print(f"{'Kendall (orig)':25s} {np.mean(t_bo):8.4f} {np.percentile(t_bo,2.5):8.4f}
↪  {np.percentile(t_bo,97.5):8.4f}")
print(f"{'Kendall (sim)':25s} {np.mean(t_bs):8.4f} {np.percentile(t_bs,2.5):8.4f}
↪  {np.percentile(t_bs,97.5):8.4f}")
print(f"{'Spearman (orig)':25s} {np.mean(r_bo):8.4f} {np.percentile(r_bo,2.5):8.4f}
↪  {np.percentile(r_bo,97.5):8.4f}")
```

```
print(f"{'Spearman (sim)':25s} {np.mean(r_bs):8.4f} {np.percentile(r_bs,2.5):8.4f}
↪  {np.percentile(r_bs,97.5):8.4f}")

# les IC se chevauchent bien => la copule Clayton reproduit la dependance


# ========================================
# FIGURES EXERCICE 5
# ========================================

# overview donnees
fig, axes = plt.subplots(2,2, figsize=(12,10))
ax=axes[0,0]; ax.plot(p1, color='steelblue', lw=0.8, label='actif 1')
ax.plot(p2, color='coral', lw=0.8, label='actif 2')
ax.set_xlabel('jours'); ax.set_ylabel('prix'); ax.set_title('prix'); ax.legend();
↪  ax.grid(True, alpha=0.3)

ax=axes[0,1]; ax.plot(lr1, color='steelblue', lw=0.5, alpha=0.7, label='actif 1')
ax.plot(lr2, color='coral', lw=0.5, alpha=0.5, label='actif 2')
ax.set_xlabel('jours'); ax.set_ylabel('log-ret'); ax.set_title('log-returns'); ax.legend();
↪  ax.grid(True, alpha=0.3)

ax=axes[1,0]
ax.hist(lr1, bins=50, density=True, alpha=0.6, color='steelblue', edgecolor='white',
↪  linewidth=0.3, label='actif 1')
xh = np.linspace(min(lr1), max(lr1), 200)
ax.plot(xh, norm.pdf(xh, np.mean(lr1), np.std(lr1)), 'b-', lw=2)
ax.hist(lr2, bins=50, density=True, alpha=0.5, color='coral', edgecolor='white',
↪  linewidth=0.3, label='actif 2')
xh2 = np.linspace(min(lr2), max(lr2), 200)
ax.plot(xh2, norm.pdf(xh2, np.mean(lr2), np.std(lr2)), 'r-', lw=2)
ax.set_xlabel('log-return'); ax.set_ylabel('densite'); ax.set_title('histogrammes');
↪  ax.legend(); ax.grid(True, alpha=0.3)

ax=axes[1,1]; ax.scatter(lr1, lr2, s=5, alpha=0.4, color='steelblue')
ax.set_xlabel('LR actif 1'); ax.set_ylabel('LR actif 2')
ax.set_title(f'scatter (corr={corr_lr:.3f})'); ax.grid(True, alpha=0.3)

plt.suptitle('Exercice 5: donnees', fontsize=14, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig('ex5_data_overview.png', dpi=150, bbox_inches='tight')
plt.close()

# QQ plots
fig, axes = plt.subplots(1,2, figsize=(10,4.5))
for idx, (lr, name) in enumerate([(lr1,'actif 1'),(lr2,'actif 2')]):
    ax = axes[idx]; sl = np.sort(lr); nq = len(sl)
    tq = norm.ppf((np.arange(1,nq+1)-0.5)/nq)
    ln = (sl - np.mean(lr))/np.std(lr)
    ax.scatter(tq, ln, s=5, alpha=0.5, color='steelblue')
    ax.plot([-4,4],[-4,4],'r--',lw=1.5)
    ax.set_xlabel('th N(0,1)'); ax.set_ylabel('emp')
    ax.set_title(f'QQ {name}'); ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('ex5_qq_plots.png', dpi=150, bbox_inches='tight')
plt.close()

# simulation
fig, axes = plt.subplots(2,2, figsize=(12,10))
ax=axes[0,0]; ax.plot(range(n_data), p1, 'steelblue', lw=0.8, label='original')
ax.plot(range(n_data, n_data+1001), p1s, 'orange', lw=0.8, label='simule')
ax.axvline(x=n_data, color='red', ls='--', alpha=0.5)
ax.set_xlabel('jours'); ax.set_ylabel('prix'); ax.set_title('actif 1'); ax.legend();
↪  ax.grid(True, alpha=0.3)
```

```
ax=axes[0,1]; ax.plot(range(n_data), p2, 'coral', lw=0.8, label='original')
ax.plot(range(n_data, n_data+1001), p2s, 'orange', lw=0.8, label='simule')
ax.axvline(x=n_data, color='red', ls='--', alpha=0.5)
ax.set_xlabel('jours'); ax.set_ylabel('prix'); ax.set_title('actif 2'); ax.legend();
↪  ax.grid(True, alpha=0.3)

ax=axes[1,0]
ax.hist(lr1s, bins=50, density=True, alpha=0.5, color='orange', edgecolor='white',
↪  linewidth=0.3, label='sim')
ax.hist(lr1, bins=50, density=True, alpha=0.5, color='steelblue', edgecolor='white',
↪  linewidth=0.3, label='orig')
ax.set_xlabel('log-ret'); ax.set_ylabel('densite'); ax.set_title('actif 1: LR'); ax.legend();
↪  ax.grid(True, alpha=0.3)

ax=axes[1,1]
ax.scatter(lr1s, lr2s, s=5, alpha=0.3, color='orange', label=f'sim (r={cs:.3f})')
ax.scatter(lr1, lr2, s=5, alpha=0.3, color='steelblue', label=f'orig (r={corr_lr:.3f})')
ax.set_xlabel('LR 1'); ax.set_ylabel('LR 2'); ax.set_title('scatter'); ax.legend(fontsize=9);
↪  ax.grid(True, alpha=0.3)
plt.suptitle('Exercice 5: simulation', fontsize=14, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig('ex5_simulation.png', dpi=150, bbox_inches='tight')
plt.close()

# bootstrap
fig, axes = plt.subplots(1,2, figsize=(10,4.5))
ax=axes[0]; ax.hist(t_bo, bins=30, density=True, alpha=0.5, color='steelblue', label='orig')
ax.hist(t_bs, bins=30, density=True, alpha=0.5, color='orange', label='sim')
ax.axvline(x=tau_k, color='blue', ls='--', lw=1.5); ax.axvline(x=tau_sim, color='orange',
↪  ls='--', lw=1.5)
ax.set_xlabel('Kendall tau'); ax.set_title('bootstrap: Kendall'); ax.legend(); ax.grid(True,
↪  alpha=0.3)

ax=axes[1]; ax.hist(r_bo, bins=30, density=True, alpha=0.5, color='steelblue', label='orig')
ax.hist(r_bs, bins=30, density=True, alpha=0.5, color='orange', label='sim')
ax.axvline(x=rho_orig, color='blue', ls='--', lw=1.5); ax.axvline(x=rho_sim, color='orange',
↪  ls='--', lw=1.5)
ax.set_xlabel('Spearman rho'); ax.set_title('bootstrap: Spearman'); ax.legend(); ax.grid(True,
↪  alpha=0.3)

plt.suptitle('Exercice 5: bootstrap', fontsize=14, fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig('ex5_bootstrap.png', dpi=150, bbox_inches='tight')
plt.close()

print("\n=> toutes les figures sauvegardees")
print("\n" + "="*60)
print("PROJET FINI !")
print("="*60)
```